



**Politecnico
di Torino**

Politecnico di Torino

Bachelor degree in Management Engineering

A.a. 2025/2026

Graduation Session ... 2026

Permutation flowshop scheduling with periodic maintenance

Supervisor:

Fabio Salassa

Candidate:

Dania Ottenga

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodos egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Tables	VI
List of Figures	VII
1 Introduction to combinatorial optimization	1
1.1 Historical roots	1
1.2 Families of problems	2
1.3 Complexity and Solving Strategies	2
1.3.1 NP-hard Problems	2
1.3.2 Heuristics and Metaheuristics	2
2 Introduction to Machine Scheduling	5
2.1 Problem classification	5
2.2 Flow shop scheduling	6
2.3 Permutation Flow Shop (PFSSP)	6
3 Problem identification	7
3.1 Mathematical Notation: Parameters, Indexes and Variables	7
3.2 Constraints and objective function	7
3.3 Modeling Maintenance in Idle Times	9
3.4 The Proposed Metaheuristic	9
4 Procedures	11
4.1 Data Management and Input	11
4.2 MIP implementation	11
4.3 Metaheuristic Implementation	14
4.3.1 Schedule operation	14
4.3.2 Compute Schedule	14
4.3.3 Compute Schedule	15
4.3.4 Swap Move	16
4.3.5 Simulated Annealing	16

4.4 Results Visualization: The Gantt Chart Generator	16
5 Results	19
6 Conclusions	23
A Scripts	25
A.1 Data Generation Script	25
A.2 MIP implementation Script	26
A.3 Simulated Annealing Script	32
Bibliography	41

List of Tables

3.1	Mathematical Notation: Parameters, Indexes and Variables	9
5.1	Computational results: Exact Algorithm vs. Metaheuristic.	19

List of Figures

4.1	Representation of the solution to the problem with the Gantt Chart	17
-----	--	----

Chapter 1

Introduction to combinatorial optimization

Combinatorial Optimization (CO) is the search for a “best” configuration of a set of discrete variables to achieve some goals in a typically finite domain. To optimize means to minimize or maximize the objective function, respecting the constraints. The solution will be a combination of variables, present in the set of all feasible assignments: the search (or solution) space.

A CO problem is defined by:

- **Set of variables:** $V = \{x_1, x_2, \dots, x_n\}$.
- **Variable domains:** $D = \{D_1, D_2, \dots, D_n\}$ where x_i can take on values ($x_i \in D_i$).
- **Constraints among variables:** Ω is the set of restrictions that limits all the possible combinations of values defining the solution space $S \subseteq D_1 \times \dots \times D_n$.
- **Objective function:** $f : S \rightarrow \mathbb{R}$ assigns a numeric value to each admissible configuration.

1.1 Historical roots

The origins of CO lie in economic problems: the planning and management of operations and the efficient use of resources. It was initially conceived by Kantorovich and Koopmans and was motivated by applications in transportation and transshipment [1]. Soon more technical applications were developed and today we see discrete optimization problems everywhere: portfolio selection, capital budgeting, design of marketing campaigns, investment planning and facility location,

political districting, gene sequencing, classification of plants and animals, the design of new molecules, the determination of ground states, layout of survivable and cost-efficient communication networks, positioning of satellites, sizing of truck fleets, layout of mass transportation systems and scheduling of buses and trains, assignment of workers to jobs such as drivers to buses and airline crew scheduling, design of unbreakable codes, etc. The list of applications is endless; even in areas like sports archaeology or psychology [2].

1.2 Families of problems

We can divide the CO in five macro-groups: **the assignment problem** that aims to find the permutation where the sum of the costs of all the variables is the smallest possible by assigning a task to each agent; **the transportation problem** that aims to minimize the cost of transporting a product between several sources and destinations, **the packing/covering problem** that aims to minimize the number of containers used or the value of the object inserted in them; **the traveling salesman problem** that aims to find the order of the given places that the salesman must visit each location exactly once, traversing the shortest route and **the scheduling problem** that aims to allocate resources to perform a set of tasks over a specific period minimizing the completion time.

1.3 Complexity and Solving Strategies

The main challenge in CO is the size of the search space S . For many problems, S grows factorially or exponentially with the number of variables n .

1.3.1 NP-hard Problems

Visiting all the possibilities in our search space to find the best one is not always computationally possible; these are known as **NP-hard problems** for which no polynomial time algorithm is currently known. This means that as the problem size increases, the time required to find the "best" configuration using exact methods becomes practically impossible, even for modern computers [3].

1.3.2 Heuristics and Metaheuristics

In these cases, we can rely on algorithms that produce an approximately optimal solution by making a trade-off between computational time and solution quality. So, the aim is to find the one solution that is close to the optimal in a reasonable time [4].

Approximate algorithms are divided into two major categories of algorithms. **Heuristics** are problem-specific algorithms designed to find a good solution quickly. They often follow a greedy approach (e.g., in the traveling salesman problem, always moving to the nearest unvisited city). While fast, heuristics often get stuck in local optima — solutions that are better than their neighbors but worse than the global best. On the other hand, **Metaheuristics** are higher-level frameworks that orchestrate one or more heuristics to explore the search space more efficiently. Unlike simple heuristics, metaheuristics include mechanisms to escape local optima (e.g., by allowing temporary "worse" moves or maintaining a population of solutions). Common examples include Genetic Algorithms, Simulated Annealing, and Ant Colony Optimization.

Chapter 2

Introduction to Machine Scheduling

Machine Scheduling aims to perform a number of jobs, each of which consists of a sequence of operations, by using a number of machines. To complete a job, each of its operations must be processed in the order given by the sequence. The processing of an operation requires a use of a particular machine for a given duration, the processing time. The objective is to find a processing order to minimize the makespan.

A distinction can be made between sequences and schedules. A **sequence** corresponds to an ordering of the operations on each machine that allows the sequence of the operations corresponding to each job. Corresponding to each sequence there is an infinity of feasible **schedules** obtained by specifying the exact starting and finishing time of each operation. A schedule determines for each job a **completion time** in which its last operation is finished [5].

2.1 Problem classification

To classify the various types of machine scheduling problems, the three field notation introduced by **Graham** is used [6] [7]:

- α for the **resources**: we can consider a single machine, parallel identical machines, or flow shop environments.
- β for the **properties of tasks**: where we can consider:
 - **Preemption**: not allowed (one cannot stop the task once it has started).
 - **Precedence**: no constraints, strict constraints (finish-start: the successor cannot start before the predecessor finishes) and generalized constraints.

- **Deadlines:** not assumed, imposed on activities and imposed on the project.
- γ for the **criterion** (the objective function) which is to minimize the makespan ($C_{\max}$), or the completion time of the last job.

2.2 Flow shop scheduling

Flow Shop is a processing system in which the task sequence of each job is fully specified (chain precedence) and all the jobs visit the work stations in the same order. In a pure flow shop each job visits all stages but more generically can have fewer tasks and can skip some stations. But it's assumed that a job visiting a $i' > i$ station cannot go back to the previous one (i). There is also another case: the reentrant flow shop. Here jobs can recycle back and be reprocessed at the same station, or sequence of stations, multiple times.

There are many variations to the general flow shop like the **simple flow shop**. Here we can make these considerations: each stage can handle one operation at a time, each task of a job requires a different machine, all jobs are available simultaneously at the start and remain available until the work is finished, each job can be in one place at the time, no preemption is allowed, when a job is finished in one machine, it's immediately available for the next one so there are no delays due to setups or transfer, intermediate storage between machines is unlimited. Then there is the **general job shop**, which is described exactly like the simple flow shop but the machine sequence of different jobs may differ, and the **hybrid (compound or flexible) flow shop** where work station may consists in several functionally identical processors in parallel [8].

2.3 Permutation Flow Shop (PFSSP)

The **Permutation Flow Shop Scheduling Problem** (PFSSP) consists of a set of jobs on a set of machines with the objective of minimizing the makespan. Each job can be processed on one machine at a time and each machine can process one job at a time and the operations can't be preemptable. The sequence in which the jobs are to be processed is the same for each machine.

The PFSSP problem is denoted by the symbols $n/m/P/C_{\max}$ where n represents the number of jobs, m the number of machines, P is the processing time and $C_{\max}$ is the makespan. Based on the considerations exposed, the **PFSSP** model will be used for the case study analysis [9].

Chapter 3

Problem identification

The problem addressed in this study takes into consideration a permutation flowshop scheduling with periodic maintenance. The aim is, given the number of jobs, the number of machines and the processing time of every job on every machine, to minimize the completion time of the final job in the sequence. The production will be carried out within working shifts. To reduce time wasting given by the transfer of duties and the explanation of the work carried out up to the end of the shift, we will consider the end of the shift as a time limit to finish the job we've started so there are no jobs that start in a shift and end in another. So when deciding whether to start a job during we see that there isn't enough time to complete the job in the current shift, we will wait and start producing it in the next one.

3.1 Mathematical Notation: Parameters, Indexes and Variables

To transition from the description of the problem to the mathematical model, it is necessary to define the notation used to represent the production environment and the scheduling constraints [10]. It is explained in more detail in the table {3.1}.

3.2 Constraints and objective function

Objective function

$$\min E_{mn} \tag{3.1}$$

Eq. (3.1) states the objective to be minimized (makespan), defined as the completion time in the last machine (m) of the job in the last position (n).

Costraints

$$\sum_{l=1}^n Z_{jl} = 1 \quad 1 \leq j \leq n \quad (3.2)$$

$$\sum_{j=1}^n Z_{jl} = 1 \quad 1 \leq l \leq n \quad (3.3)$$

Eqs. (3.2) force that each job is assigned to only one position in the sequence, while Eqs. (3.3) ensure that each position is occupied by only one job.

$$E_{il} + \sum_{j=1}^n p_{ij} Z_{jl+1} \leq E_{il+1} \quad 1 \leq i \leq m, \quad 1 \leq l \leq n-1 \quad (3.4)$$

$$E_{il} + \sum_{j=1}^n p_{i+1j} Z_{jl} \leq E_{i+1l} \quad 1 \leq i \leq m-1, \quad 1 \leq l \leq n \quad (3.5)$$

Eqs. (3.4) force that, for each machine, two consecutive jobs do not overlap their processing, while Eqs. (3.5) impose that operations of a job in two consecutive machines do not overlap.

$$E_{11} \geq \sum_{j=1}^n p_{1j} Z_{j1} \quad (3.6)$$

Eq. (3.6) computes the completion time of the job scheduled in the first position in the first machine.

$$E_{il} - sT \leq M(1 - \gamma_{ils}) \quad 1 \leq i \leq m \quad 1 \leq l \leq n, \quad 1 \leq s \leq S \quad (3.7)$$

$$E_{il} - \sum_{j=1}^n p_{ij} Z_{jl} + M(1 - \gamma_{ils}) \geq T(s-1) \quad 1 \leq i \leq m, \quad 1 \leq l \leq n, \quad 1 \leq s \leq S \quad (3.8)$$

Sets of Eqs. (3.7) and (3.8) control that each operation starting in a shift also finishes in within this shift.

$$\sum_{s=1}^S \gamma_{ils} = 1 \quad 1 \leq i \leq m, \quad 1 \leq l \leq n \quad (3.9)$$

Finally, the set of Eqs. (3.9) ensures that each operation is scheduled in one shift.

Table 3.1: Mathematical Notation: Parameters, Indexes and Variables

Symbol	Description
<i>Parameters</i>	
n	Number of jobs
m	Number of machines
$p_{i,j}$	Processing time of job j on machine i
T	Length of the shifts
M	Big number
S	Maximal number of shifts
<i>Indexes</i>	
j	Jobs $1 \leq j \leq n$
i	Machines $1 \leq i \leq m$
l	Positions $1 \leq l \leq n$
s	Shifts $1 \leq s \leq S$
<i>Variables</i>	
E_{ij}	Completion time of job in position j on machine i
Z_{jl}	$\begin{cases} 1 & \text{if job } j \text{ is in position } l \\ 0 & \text{otherwise} \end{cases}$
γ_{ils}	$\begin{cases} 1 & \text{if job in position } l \text{ is in shift } s \text{ on machine } i \\ 0 & \text{otherwise} \end{cases}$

3.3 Modeling Maintenance in Idle Times

In this particular study, we consider the **idle time** (the time between the finish of a job on a machine and the start of the next one on the same machine) as a maintenance opportunity. This approach aims to exploit natural gaps in the schedule, performing maintenance without interrupting active processing, thus preserving productivity.

3.4 The Proposed Metaheuristic

Metaheuristics have been established as one of the most practical approaches to simulation optimization. They are designed to tackle complex optimization problems where other optimization methods have failed to be either effective or efficient. A good metaheuristic implementation is likely to provide near optimal

solutions in reasonable computation times [11]. In this study we will apply the **Simulated Annealing** (SA) metaheuristic.

Simulated annealing is a probabilistic method proposed in Kirkpatrick, Gelett and Vecchi (1983) and Cerny (1985) for finding the global minimum of a cost function that may possess several local minima [12]. In condensed matter physics, annealing denotes a physical process in which a solid in a heat bath is heated up by increasing the temperature of the heat bath to a maximum value at which all particles of the solid randomly arrange themselves in the liquid phase, followed by cooling through slowly lowering the temperature of the heat bath. In this way, all particles arrange themselves in the low energy ground state of the corresponding lattice, provided the maximum temperature is sufficiently high and the cooling is carried out sufficiently slowly [13].

Chapter 4

Procedures

4.1 Data Management and Input

To create the inputs values, such as the number of jobs, the number of machines and the processing time for job for machine, a specific script was developed. The function of the script is to create 50 instances, in the folder of the project, to be then applied in the exact model and in the metaheuristic to analyze the differences in the results.

The logic of data creation is explained in the **Algorithm 1**¹.

4.2 MIP implementation

The optimal solution will be calculated using the python programming language, in particular the Python-MIP package. **Python-MIP** is a collection of Python tools for the modeling and solution of Mixed-Integer Linear programs (MIPs)².

In the **Algorithm 2**, the script access the values of the number of jobs, the number of machines and the processing times from the file the users provides as input and calculates the Big-M value, the shift length and the maximal number of shifts using the data retrieved before:

Then the variables, the objective function and the constraints will be generated, in the same way as they were discussed in the third chapter. Then the model can optimize the function and give us a value. A timer is set on 300 seconds. If the computation exceeds the timer set, there are two possibilities: a feasible solution is found or no solution is found.

¹For the full source code, please refer to Appendix A.1.

²For the full source code, please refer to Appendix A.2.

Algorithm 1 MIP Algorithm

```

1: for  $i \leftarrow 1$  to 50 do                                ▷ Generates 50 different instances
2:    $n \leftarrow \text{RandomInteger}(5, 20)$                     ▷ Generates the number of jobs
3:    $m \leftarrow \text{RandomInteger}(5, 10)$                   ▷ Generates the number of machines

4:    $\text{CreateFile}(\text{Instance\_}i)$ 
5:    $\text{WriteToFile}(n, m)$                                 ▷ Writes dimensions on the first line

6:   for  $l \leftarrow 1$  to  $m$  do
7:      $\text{Row} \leftarrow []$ 
8:     for  $i \leftarrow 1$  to  $n$  do
9:        $p_{ij} \leftarrow \text{RandomInteger}(1, 100)$           ▷ Generates the processing time
10:       $\text{Append}(\text{Row}, p_{ij})$ 
11:    end for
12:     $\text{WriteToFile}(\text{Row})$ 
13:  end for

14:   $\text{CloseFile}(\text{Instance\_}i)$ 
15: end for

```

Algorithm 2 Parameters

```
1:  $File \leftarrow InputValue$ 

2: if  $FileExists$  then
3:    $OpenFile(File)$ 

4:    $i \leftarrow 1$ 
5:   for all  $Line \in File$  do
6:     if  $i = 0$  then
7:        $n \leftarrow \text{First element of } Line$   $\triangleright$  Gets the number of jobs
8:        $m \leftarrow \text{Second element of } Line$   $\triangleright$  Gets the number of machines
9:     else
10:       $Times \leftarrow []$ 
11:      for all  $Value \in Line$  do
12:         $Append(Times, Value)$   $\triangleright$  Gets the processing times
13:      end for
14:       $Append(p_{ij}, Times)$ 
15:    end if
16:     $i \leftarrow i + 1$ 
17:  end for

18: else
19:   stop
20: end if

21:  $M \leftarrow 0$ 
22:  $MaxProcessingTime \leftarrow 0$ 

23: for  $p \in p_{ij}$  do
24:    $M \leftarrow M + p$   $\triangleright$  Calculates the big M
25:    $MaxProcessingTime \leftarrow Max(MaxProcessingTime, p)$ 
26: end for

27:  $T \leftarrow MaxProcessingTime * RandomInteger(2, 5)$   $\triangleright$  Calculates the shift length

28:  $S \leftarrow Max(1, \lceil M/T \rceil)$   $\triangleright$  Calculates the maximal number of shifts
```

4.3 Metaheuristic Implementation

The input creation part is the same as the previous one. The Big-M value will be generated in the same way but the shift length and the maximal number of shifts will be given in input from the user calculated in the previous test on the MIP implementation. To initialize the algorithm the starting temperature is values at 1000, the cooling rate at 0.9998 and the maximum number of iterations at 100000. The functions of the method are defined as follows³.

4.3.1 Schedule operation

Algorithm 3 schedules a single operation respecting the shifts. First it states the earliest shift in which the job can start. If the job can be scheduled in the earliest shift it will finish also there otherwise it'll start and finish in the next shift.

Algorithm 3 Schedule operation

```

1: function SCHEDULEOPERATION(EarliestStart, ProcessingTime, T)

2:   Shift  $\leftarrow$  Integer(EarliestStart/T)

3:   if EarliestStart + ProcessingTime  $\leq$  (Shift + 1) * T then
4:     return EarliestStart + ProcessingTime
5:   else
6:     return (Shift + 1) * T + ProcessingTime
7:   end if
8: end function

```

4.3.2 Compute Schedule

Algorithm 4 computes the E matrix given a permutation list. First it selects the number of the job. If the first machine and the first job are chosen, the earliest start equals zero. If the first machine but not the first job are chosen, we take the completion time of the job before in the same machine. If the first job but not the first machine are chosen, we take the completion time of the first job of the machine before. If neither the first machine or the first job are chosen, we take the biggest between the completion time of the job before in the same machine and of

³For the full source code, please refer to Appendix A.3.

the same job in the machine before. Knowing now the earliest start we schedule the operation.

Algorithm 4 Compute Schedule

```

1: function COMPUTESCHEDULE(PermutationList)

2:   for  $i \leftarrow 1$  to  $m$  do
3:     for  $l \leftarrow 1$  to  $n$  do

4:        $j \leftarrow \text{PermutationList}[l]$ 

5:       if  $i = 0$  and  $l = 0$  then
6:          $\text{EarliestStart} \leftarrow 0$ 
7:       else if  $i = 0$  and  $l > 0$  then
8:          $\text{EarliestStart} \leftarrow E_{0,l-1}$ 
9:       else if  $i > 0$  and  $l = 0$  then
10:         $\text{EarliestStart} \leftarrow E_{i-1,0}$ 
11:       else
12:         $\text{EarliestStart} \leftarrow \max(E_{i-1,l}, E_{i,l-1})$ 
13:       end if

14:         $E_{i,l} \leftarrow \text{SCHEDULEOPERATION}(\text{EarliestStart}, p_{i,j}, T)$ 
15:     end for
16:   end for

17:   return  $E$ 
18: end function

```

4.3.3 Compute Schedule

Algorithm 5 gives the total makespan of the job shop.

Algorithm 5 Makespan

```

1: function MAKESPAN(PermutationList)

2:    $E \leftarrow \text{COMPUTESCHEDULE}(\text{PermutationList})$ 
3:   return  $E_{m-1,n-1}$ 
4: end function

```

4.3.4 Swap Move

Algorithm 6 swaps two random jobs to see if the solution is better or worse.

Algorithm 6 Swap Move

```

1: function SWAPMOVE(PermutationList)

2:   Select two random indexes  $i, j \in \{1, \dots, |PermutationList|\}$ 
3:   return  $E_{m-1, n-1}$ 
4:    $PermutationList[i], PermutationList[j] \leftarrow PermutationList[j], PermutationList[i]$ 
5:   return PermutationList
6: end function

```

4.3.5 Simulated Annealing

Algorithm 7 Simulated annealing

▷ We select the initial solution sorted by descending values of the sum of the processing time for every job

```

1: current_list  $\leftarrow$  jobs  $\{1, \dots, n\}$  sorted in descending order of  $\sum_{i=1}^m p_{ij}$ 

```

4.4 Results Visualization: The Gantt Chart Generator

For the visualization of the results it was used the python library “matplotlib” to generate a **Gantt Chart**. On the x -axis the time is displayed and on the y -axis the machines are shown. The planning starts from the first machine on top and procedes downwards to the last. Every job is indicated with the j_n notation and has a specific color to distinguish from the others. The idle times are yellow colored with oblique lines and the text “maintenance”.

In figure 4.1 is presented a simple example.

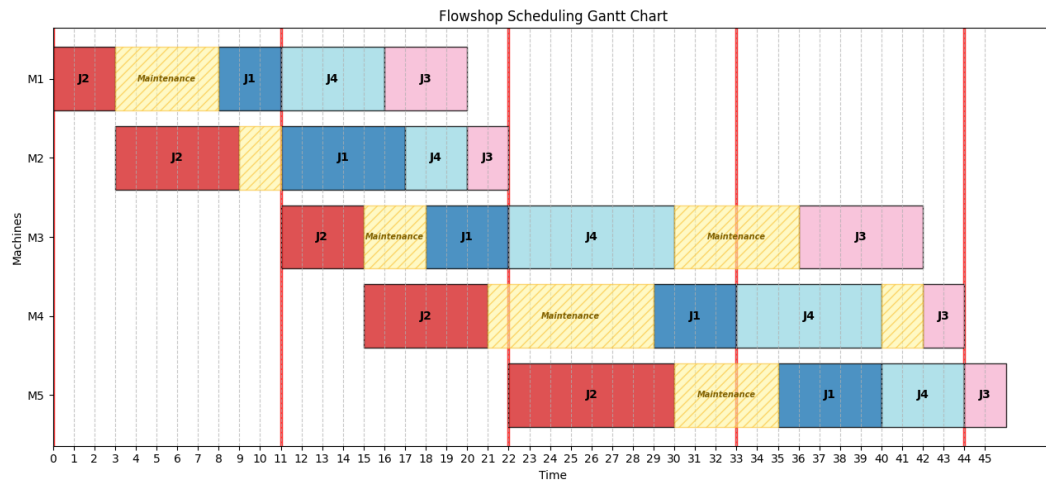


Figure 4.1: Representation of the solution to the problem with the Gantt Chart

Chapter 5

Results

To rigorously evaluate the performance of the metaheuristic, two distinct percentage gaps are computed:

- **Gap vs Best:** Measures the deviation of the metaheuristic solution from the best feasible solution (Upper Bound) found by the exact algorithm.

$$\text{Gap}_{\text{Best}}(\%) = \frac{\text{Obj}_{\text{Meta}} - \text{Obj}_{\text{MIP}}}{\text{Obj}_{\text{MIP}}} \times 100 \quad (5.1)$$

- **Gap vs LB (Worst-case Optimality Gap):** Measures the maximum possible deviation from the theoretical optimum, using the Lower Bound provided by the MIP solver. This metric is particularly useful for instances where the exact algorithm reaches the time limit without proving optimality.

$$\text{Gap}_{\text{LB}}(\%) = \frac{\text{Obj}_{\text{Meta}} - \text{LB}_{\text{MIP}}}{\text{LB}_{\text{MIP}}} \times 100 \quad (5.2)$$

For instances solved to exact optimality within the time limit, the Best Objective and the Lower Bound coincide, rendering the two gap values identical.

Table 5.1: Computational results: Exact Algorithm vs. Metaheuristic.

Inst.	Exact Alg. (MIP)			Metaheuristic		Gaps (%)	
	LB	Best Obj.	Time (s)	Obj.	Time (s)	vs Best	vs LB
Instance_01	1254	-	300*	1293	1.5825	-	3.11
Instance_02	831	1036	300*	893	0.6973	-13.80	7.46
Instance_03	987	1153	300*	1117	1.0735	-3.12	13.17
Instance_04	1383	-	300*	1539	2.1735	-	11.28

Continues in the next page...

Table 5.1 – Continuation from the previous page

Inst.	Exact Alg. (MIP)			Metaheuristic		Gaps (%)	
	LB	Best Obj.	Time (s)	Obj.	Time (s)	vs Best	vs LB
Instance_05	616	616	5.5781	616	0.6787	0.0	0.0
Instance_06	1392	-	300*	1587	2.8429	-	13.97
Instance_07	757	757	265.3711	757	0.7486	0.0	0.0
Instance_08	751	751	14.1740	751	1.0578	0.0	0.0
Instance_09	1092	-	300*	1155	1.8492	-	5.77
Instance_10	931	-	300*	1075	1.7682	-	15.47
Instance_11	922	922	296.0627	922	1.0600	0.0	0.0
Instance_12	815	-	300*	965	1.3909	-	18.40
Instance_13	865	-	300*	909	1.3276	-	5.09
Instance_14	1135	-	300*	1198	2.1418	-	5.55
Instance_15	258763715079	-	300*	1333	2.3519	-	-100
Instance_16	1243	1347	300*	1243	1.6026	-7.72	0.00
Instance_17	1144	-	300*	1193	2.0699	-	4.28
Instance_18	918	918	22.0443	918	1.0590	0.0	0.0
Instance_19	955	-	300*	997	1.5755	-	4.33979
Instance_20	1098	-	300*	1218	2.6344	-	10.97
Instance_21	1358	-	300*	1378	1.6639	-	1.47
Instance_22	861	-	300*	957	1.4389	-	11.15
Instance_23	1066	-	300*	1222	1.8198	-	14.63
Instance_24	589	589	14.5636	589	0.7617	0.0	0.0
Instance_25	1207	-	300*	1265	1.7439	-	4.81
Instance_26	749	885	300*	869	1.2907	-1.81	16.02
Instance_27	706	905	300*	839	0.9046	-7.2928	18.84
Instance_28	784	-	300*	968	1.3762	-	23.4694
Instance_29	823	3544	300*	1028	1.3638	-70.99	24.91
Instance_30	639	639	5.7913	639	0.7817	0.0	0.0
Instance_31	728	2495	300*	854	0.9952	-65.77	17.31
Instance_32	659	659	29.2018	659	0.6788	0.0	0.0
Instance_33	939	-	300*	1089	1.6756	-	15.97
Instance_34	581	581	19.1375	581	0.7810	0.0	0.0
Instance_35	985	-	300*	1083	1.5607	-	9.95
Instance_36	759	1173	300*	845	1.1697	-27.96	11.33
Instance_37	1217	-	300*	1392	2.8524	-	14.4218
Instance_38	1025	5485	300*	1242	1.4063	-77.36	21.17
Instance_39	1107	1143	300*	1132	1.2764	-0.96	2.24
Instance_40	714	714	31.4167	714	0.8106	0.0	0.0
Instance_41	697	799	300*	731	0.8083	-8.51	4.88
Instance_42	926	-	300*	1111	1.3021	-	19.98
Instance_43	1037	4078	300*	1062	1.1154	-73.96	2.4108
Instance_44	867	-	300*	988	1.2438	-	13.96
Instance_45	1418	-	300*	1644	2.4095	-	15.94
Instance_46	764	878	300*	876	0.9081	-0.2279	14.66
Instance_47	1019	-	300*	1222	1.4539	-	19.92
Instance_48	645	645	7.1077	645	0.5621	0.0	0.0

Continues in the next page...

Table 5.1 – Continuation from the previous page

Inst.	Exact Alg. (MIP)			Metaheuristic		Gaps (%)	
	LB	Best Obj.	Time (s)	Obj.	Time (s)	vs Best	vs LB
Instance_49	879	879	237.48057985305786	879	0.8228	0.0	0.0
Instance_50	908	-	300*	936	1.3186	-	3.08

* Best feasible solution found. Time limit reached.

farlo in csv

Chapter 6

Conclusions

Appendix A

Scripts

A.1 Data Generation Script

```
1 import os
2 import random
3
4
5 def creation_files():
6     directory = os.path.dirname(os.path.abspath(__file__))
7
8     for i in range(1, 51):
9         job = random.randint(5, 20)
10        machine = random.randint(5, 10)
11
12        file_name = f"instance_{i}.txt"
13        path = os.path.join(directory, file_name)
14
15        with open(path, "w", encoding="utf-8") as f:
16            f.write(f"{job} {machine}\n")
17
18            for _ in range(machine):
19                processing_times = [str(random.randint(1,
20                100)) for _ in range(job)]
21                f.write(" ".join(processing_times) + "\n")
22
23 creation_files()
24 print("The files have been created!")
```

creation_files.py

A.2 MIP implementation Script

```

1 import math
2 import os
3 import random
4 from itertools import product
5 from mip import *
6 import matplotlib.pyplot as plt
7 import pandas as pd
8
9
10 # =====
11 ''' PARAMETERS '''
12 # =====
13
14 p = []
15 file = input('Define the name of your input file: ')
16 if os.path.exists(file):
17     print(f"File '{file}' loaded successfully!")
18     with open(file, 'r', encoding = 'utf-8') as f:
19         for i, line in enumerate(f):
20             if i == 0:
21                 values = line.split()
22
23                 # Number of jobs
24                 n = int(values[0])
25
26                 # Number of machines
27                 m = int(values[1])
28
29             else:
30                 values = line.split()
31
32                 # Processing time of job j in machine i (p
33                 ij)
34                 l = []
35                 for j in range(len(values)):
36                     l.append(int(values[j]))
37                 p.append(l)
38 else:
39     print(f"Error: File '{file}' doesn't exist.")
40     exit()
41

```

```

42 # Big number
43 M = 0
44 max_p_t = 0
45 for row in p:
46     for element in row:
47         M += element
48         max_p_t = max(max_p_t, element)
49
50
51 # Shift length
52 T = max_p_t * random.choice([2, 3, 4, 5])
53
54 # Maximal number of shifts
55 S = max(1, math.ceil(M/T))
56
57 print(f"Big number created: {M}\n"
58       f"Shift length created: {T}\n"
59       f"Maximal number of shifts created: {S}\n")
60
61 mod = Model("Permutation flowshop scheduling") # sense =
        MINIMIZE
62
63 # =====
64 ''' VARIABLES '''
65 # =====
66
67 # Completion time of job in machine i in position l (E il)
68 E = [[mod.add_var(name="E({},{})".format(i, l), var_type =
        INTEGER)
69         for l in range(n)]
70         for i in range(m)]
71
72 # If job j is scheduled in position l (Z jl)
73 Z = [[mod.add_var(name="Z({},{})".format(j, l), var_type=
        BINARY)
74         for l in range(n)]
75         for j in range(n)]
76
77 # If job in position l is scheduled in the shift s in
        machine i (gamma ils)
78 gamma = [[[mod.add_var(name="gamma({},{},{})".format(i, l, s
        ), var_type=BINARY)
79             for s in range(S)]
80             for l in range(n)]
81             for i in range(m)]

```

```

82
83 # =====
84 ''' MODEL '''
85 # =====
86
87 # Objective function
88 mod.objective = minimize(E[m - 1][n - 1])
89
90 # Each job assigned to one position
91 for j in range(n):
92     mod += xsum(Z[j][l] for l in range(n)) == 1
93
94 # Each position assigned to one job
95 for l in range(n):
96     mod += xsum(Z[j][l] for j in range(n)) == 1
97
98 # For each machine two consecutive jobs don't overlap their
   processes
99 for (i, l) in product (range(m), range(n - 1)):
100     mod += E[i][l] + xsum(p[i][j] * Z[j][l + 1] for j in
   range(n)) <= E[i][l + 1]
101
102 # Operations of a job in two consecutive machines do not
   overlap
103 for (i, l) in product (range(m - 1), range(n)):
104     mod += E[i][l] + xsum(p[i + 1][j] * Z[j][l] for j in
   range(n)) <= E[i + 1][l]
105
106 # Completion time of the job scheduled in the first
   position in the first machine
107 mod += E[0][0] >= xsum(p[0][j] * Z[j][0] for j in range(n))
108
109 # Each operation that starts in a shift finishes in the same
110 for (i, l, s) in product (range(m), range(n), range(S)):
111     mod += E[i][l] - (s + 1) * T <= M * (1 - gamma[i][l][s])
112
113     mod += E[i][l] - xsum(p[i][j] * Z[j][l] for j in range(n)
   )) + M * (1 - gamma[i][l][s]) >= T * s
114
115 # Each operation scheduled in one shift
116 for (i, l) in product (range(m), range(n)):
117     mod += xsum(gamma[i][l][s] for s in range(S)) == 1
118
119 # =====
120 ''' OPTIMIZATION '''

```

```

121 # =====
122
123 mod.emphasis = SearchEmphasis.FEASIBILITY
124 mod.max_gap = 0.00
125 status = mod.optimize(max_seconds=300)
126
127 complection_time_dict = {}
128 complection_time_list = []
129 scheduling = []
130 shifts_n_machines = []
131
132 if status == OptimizationStatus.OPTIMAL:
133     print('\nOptimal solution cost {} found'.format(mod.
134         objective_value))
135
136 elif status == OptimizationStatus.FEASIBLE:
137     print('\nSol.cost {} found, best possible: {}'.format(
138         mod.objective_value, mod.objective_bound))
139
140 elif status == OptimizationStatus.NO_SOLUTION_FOUND:
141     print('\nNo feasible solution found, lower bound is: {}'.
142         format(mod.objective_bound))
143
144 if status == OptimizationStatus.OPTIMAL or status ==
145     OptimizationStatus.FEASIBLE:
146     print('\nSolution:')
147     for v in mod.vars:
148         if abs(v.x) > 1e-6: # only printing non-zeros
149             print('{} : {}'.format(v.name, v.x))
150
151         if v.name[0] == 'E':
152             complection_time_dict[v.name] = v.x
153             complection_time_list.append(v.name)
154         elif v.name[0] == 'Z':
155             scheduling.append(v.name)
156         else:
157             shifts_n_machines.append(v.name)
158
159     print(complection_time_dict)
160     print(scheduling)
161     print(shifts_n_machines)
162
163 # =====
164 ''' GANTT CHART '''
165 # =====

```

```

162
163 if mod.num_solutions:
164
165     gantt_data = []
166     assignements = {}
167     for l in range(n):
168         for j in range(n):
169             if Z[j][l].x == 1.0:
170                 assignements[l] = j
171
172     for i in range(m):
173         for l in range(n):
174             job_idx = assignements[l]
175             end = E[i][l].x
176             duration = p[i][job_idx]
177             start = end - duration
178
179             gantt_data.append({
180                 "Machine": f"M{i + 1}",
181                 "Start": start,
182                 "End": end,
183                 "Duration": duration,
184                 "Job": f"J{job_idx + 1}"
185             })
186
187     df = pd.DataFrame(gantt_data)
188     fig, ax = plt.subplots(figsize=(15, 6))
189     cmap = plt.get_cmap('tab20')
190     job_colors = {f"J{j+1}": cmap(j) for j in range(n)}
191
192     for i, row in df.iterrows():
193         ax.barh(y=row["Machine"],
194                width=row["Duration"],
195                left=row["Start"],
196                color=job_colors[row["Job"]],
197                edgecolor='black',
198                alpha=0.8)
199
200         ax.text(x=row["Start"] + row["Duration"] / 2,
201                y=row["Machine"],
202                s=row["Job"],
203                va='center', ha='center', color='black',
204                fontweight='bold')
205
206     max_time = int(df["End"].max())

```

```

206 ax.set_xticks(range(0, max_time + 10, 10))
207 ax.set_xlabel('Time')
208 ax.set_ylabel('Machines')
209 ax.set_title('Flowshop Scheduling Gantt Chart')
210 ax.grid(True, axis='x', linestyle='--', alpha=0.7)
211
212 for s in range(S + 1):
213     shift_position = s * T
214     if shift_position < max_time:
215         ax.axvline(x=shift_position,
216                   color='red',
217                   linestyle='-',
218                   linewidth=2.5,
219                   alpha=0.8,
220                   zorder=0)
221
222 for m_id in df["Machine"].unique():
223     m_df = df[df["Machine"] == m_id].sort_values("Start"
224 )
225
226     for i in range(len(m_df) - 1):
227         actual_end = m_df.iloc[i]["End"]
228         next_start = m_df.iloc[i + 1]["Start"]
229         duration_gap = next_start - actual_end
230
231         if duration_gap > 1e-3:
232             ax.barh(y=m_id,
233                   width=duration_gap,
234                   left=actual_end,
235                   color='#FFF59D',
236                   hatch='///',
237                   edgecolor='#FBC02D',
238                   alpha=0.6,
239                   zorder=2)
240
241         if duration_gap > 50:
242             ax.text(x=actual_end + duration_gap / 2,
243                   y=m_id,
244                   s="Maintenance",
245                   va='center', ha='center',
246                   color='#7F6000',
247                   fontsize=7,
248                   fontweight='bold',
249                   fontstyle='italic')

```

```

250     ax.invert_yaxis()
251     plt.tight_layout()
252     plt.show()
253
254 else:
255     print('No solution found')

```

main.py

A.3 Simulated Annealing Script

```

1  import math
2  import os
3  import copy
4  import random
5  import time
6  import pandas as pd
7  from matplotlib import pyplot as plt
8
9
10 # =====
11 ''' PARAMETERS '''
12 # =====
13
14 p = []
15 file = input('Define the name of your input file: ')
16 if os.path.exists(file):
17     print(f"File '{file}' loaded successfully!")
18     with open(file, 'r', encoding = 'utf-8') as f:
19         for i, line in enumerate(f):
20             if i == 0:
21                 values = line.split()
22
23                 # Number of jobs
24                 n = int(values[0])
25
26                 # Number of machines
27                 m = int(values[1])
28
29             else:
30                 values = line.split()
31
32                 # Processing time of job j in machine i (p
33                 ij)

```

```

33         l = []
34         for j in range(len(values)):
35             l.append(int(values[j]))
36         p.append(l)
37
38     else:
39         print(f"Error: File '{file}' doesn't exist.")
40         exit()
41
42     # Big number
43     M = 0
44     max_p_t = 0
45     for row in p:
46         for element in row:
47             M += element
48             max_p_t = max(max_p_t, element)
49
50     # Shift length
51     T = int(input("Shift length: "))
52
53     # Maximal number of shifts
54     S = int(input("Maximal number of shifts: "))
55
56     print(f"Big number created: {M}\n")
57
58     # =====
59     ''' FUNCTIONS '''
60     # =====
61
62     def schedule_operation(earliest_start, processing_time, T):
63
64         # Earliest shift in which the job can start (starting
65         with 0)
66         shift = int(earliest_start // T)
67
68         # If the job can be scheduled in the earliest shift it
69         will finish also there
70         if earliest_start + processing_time <= (shift + 1) * T:
71             return earliest_start + processing_time
72
73         # Otherwise it'll start and finish in the next shift
74         else:
75             return (shift + 1) * T + processing_time

```



```

76 # It computes the E matrix given a permutation list
77 def compute_schedule(permutation_list):
78
79     # Completion time of job in machine i in position l (E[i][l])
80     E = [[0.0] * n for _ in range(m)]
81
82     for i in range(m):
83         for l in range(n):
84             # Number of the job
85             j = permutation_list[l]
86
87             # If it's the first machine and the first job
88             if i == 0 and l == 0:
89                 earliest_start = 0
90
91             # If it's the first machine but not the first
job we take the completion time of the job before in
92             # the same machine
93             elif i == 0 and l > 0:
94                 earliest_start = E[0][l - 1]
95
96             # If it's the first job but not the first
machine we take the completion time of the first job of
97             # the machine before
98             elif i > 0 and l == 0:
99                 earliest_start = E[i - 1][0]
100
101             # If it's not the first machine and not the
first job we take the biggest between the completion
102             # time of the job before in the same machine and
of the same job in the machine before
103             else:
104                 earliest_start = max(E[i - 1][l], E[i][l -
105 1])
106
107             # Knowing now the earliest start we schedule the
operation
108             E[i][l] = schedule_operation(earliest_start, p[i
109 ][j], T)
110
111     return E
112
113 # It gives the total makespan of the job shop

```

```

113 def makespan(permutation_list):
114     E = compute_schedule(permutation_list)
115     return E[m - 1][n - 1]
116
117
118 # It swaps two jobs to see if the solution is better or worse
119 def swap_move(permutation_list):
120     copy_list = copy.deepcopy(permutation_list)
121
122     # We select two positions of the list swapping
123     positions_swapping = random.sample(range(len(copy_list)),
124                                         2)
125
126     # And we swap
127     copy_list[positions_swapping[0]], copy_list[
128         positions_swapping[1]] = (
129         copy_list[positions_swapping[1]], copy_list[
130         positions_swapping[0]])
131     return copy_list
132
133
134 # It inserts a job in a different position to see if the solution is better or worse
135 def insertion_move(permutation_list):
136     copy_list = copy.deepcopy(permutation_list)
137
138     # We select two positions of the list inserting
139     positions_insert = random.sample(range(len(copy_list)),
140                                       2)
141
142     # And we insert
143     elem = copy_list.pop(positions_insert[0])
144     copy_list.insert(positions_insert[1], elem)
145     return copy_list
146
147
148 # It takes on a 50/50 possibility the swap or the insertion move
149 def get_neighbour(permutation_list):
150     choice = random.randint(0, 1)
151     if choice == 0:
152         return swap_move(permutation_list)
153     else:
154         return insertion_move(permutation_list)

```

```

151
152
153 def simulated_annealing(temperature, alpha, max_iterations):
154
155     # We select the initial solution sorted by descending
values of the sum of the processing time for every job
156     current_list = sorted(range(n), key=lambda j: sum(p[i][j]
157 ] for i in range(m)), reverse=True)
158     current_cost = makespan(current_list)
159
160     # The best actual solution is the initial solution
161     best_list = current_list[:]
162     best_cost = current_cost
163
164     for iteration in range(max_iterations):
165
166         # Creates a neighbour to the solution using swapping
or insert
167         neighbour_list = get_neighbour(current_list)
168         neighbour_cost = makespan(neighbour_list)
169
170         # Cost difference between the actual and the
neighbour cost
171         delta = neighbour_cost - current_cost
172
173         # If the neighbour is better we always accept it,
else we accept with probability  $e^{(-\delta/T)}$ 
174         if delta <= 0 or random.random() < math.exp(-delta /
175 temperature):
176             current_list = neighbour_list
177             current_cost = neighbour_cost
178
179         # If the current solution is better than the best
solution we update it
180         if current_cost < best_cost:
181             best_list = current_list[:]
182             best_cost = current_cost
183
184         # Geometric cooling
185         temperature *= alpha
186
187         # If temperature is too low we stop the iteration
188         if temperature < 1e-3:
189             break

```

```

189     return best_list, best_cost
190
191
192     # =====
193     ''' ALGORITHM '''
194     # =====
195
196     start = time.time()
197     best_list, best_cost = simulated_annealing(1000, 0.9998,
198         100000)
199     end = time.time()
200     print(f"Best cost: {best_cost}")
201     print(end - start)
202
203     # =====
204     ''' GANTT CHART '''
205     # =====
206
207     E = compute_schedule(best_list)
208     gantt_data = []
209
210     for i in range(m):
211         for l in range(n):
212             job_idx = best_list[l]
213             end = E[i][l]
214             duration = p[i][job_idx]
215             start = end - duration
216
217             gantt_data.append({
218                 "Machine": f"M{i + 1}",
219                 "Start": start,
220                 "End": end,
221                 "Duration": duration,
222                 "Job": f"J{job_idx + 1}"
223             })
224
225     df = pd.DataFrame(gantt_data)
226     fig, ax = plt.subplots(figsize=(15, 6))
227     cmap = plt.get_cmap('tab20')
228     job_colors = {f"J{j+1}": cmap(j) for j in range(n)}
229
230     for i, row in df.iterrows():
231         ax.barh(y=row["Machine"],
232             width=row["Duration"],
233             left=row["Start"],

```

```

233         color=job_colors[row["Job"]],
234         edgecolor='black',
235         alpha=0.8)
236
237     ax.text(x=row["Start"] + row["Duration"] / 2,
238            y=row["Machine"],
239            s=row["Job"],
240            va='center', ha='center', color='black',
241            fontweight='bold')
242
243 max_time = int(df["End"].max())
244 ax.set_xticks(range(0, max_time + 10, 10))
245 ax.set_xlabel('Time')
246 ax.set_ylabel('Machines')
247 ax.set_title('Flowshop Scheduling Gantt Chart')
248 ax.grid(True, axis='x', linestyle='--', alpha=0.7)
249
250 for s in range(S + 1):
251     shift_position = s * T
252     if shift_position < max_time:
253         ax.axvline(x=shift_position,
254                   color='red',
255                   linestyle='-',
256                   linewidth=2.5,
257                   alpha=0.8,
258                   zorder=0)
259
260 for m_id in df["Machine"].unique():
261     m_df = df[df["Machine"] == m_id].sort_values("Start")
262
263     for i in range(len(m_df) - 1):
264         actual_end = m_df.iloc[i]["End"]
265         next_start = m_df.iloc[i + 1]["Start"]
266         duration_gap = next_start - actual_end
267
268         if duration_gap > 1e-3:
269             ax.barh(y=m_id,
270                    width=duration_gap,
271                    left=actual_end,
272                    color='#FFF59D',
273                    hatch='///',
274                    edgecolor='#FBC02D',
275                    alpha=0.6,
276                    zorder=2)

```

```
277         if duration_gap > 50:
278             ax.text(x=actual_end + duration_gap / 2,
279                    y=m_id,
280                    s="Maintenance",
281                    va='center', ha='center',
282                    color='#7F6000',
283                    fontsize=7,
284                    fontweight='bold',
285                    fontstyle='italic')
286
287 ax.invert_yaxis()
288 plt.tight_layout()
289 plt.show()
```

simulated_annealing.py

Bibliography

- [1] Alexander Schrijver. «On the History of Combinatorial Optimization (Till 1960)». In: *Discrete Optimization*. Ed. by K. Aardal, G.L. Nemhauser, and R. Weismantel. Vol. 12. Handbooks in Operations Research and Management Science. Elsevier, 2005, pp. 1–68. DOI: [https://doi.org/10.1016/S0927-0507\(05\)12001-5](https://doi.org/10.1016/S0927-0507(05)12001-5). URL: <https://www.sciencedirect.com/science/article/pii/S0927050705120015> (cit. on p. 1).
- [2] Martin Grötschel and László Lovász. «Combinatorial optimization». In: *Handbook of combinatorics* 2.168 (1995), pp. 1541–1597 (cit. on p. 2).
- [3] Christian Blum and Andrea Roli. «Metaheuristics in combinatorial optimization: Overview and conceptual comparison». In: *ACM computing surveys (CSUR)* 35.3 (2003), pp. 268–308 (cit. on p. 2).
- [4] Vangelis Th Paschos. «An overview on polynomial approximation of NP-hard problems». In: *Yugoslav Journal of Operations Research* 19.1 (2009), pp. 3–40 (cit. on p. 2).
- [5] AHG Rinnooy Kan. *Machine scheduling problems: classification, complexity and computations*. Springer Science & Business Media, 2012 (cit. on p. 5).
- [6] Anna Minaeva and Zdeněk Hanzálek. «Survey on Periodic Scheduling for Time-triggered Hard Real-time Systems». In: *ACM Comput. Surv.* 54.1 (Mar. 2021). ISSN: 0360-0300. DOI: 10.1145/3431232. URL: <https://doi.org/10.1145/3431232> (cit. on p. 5).
- [7] Jan Weglarz. *Project scheduling: recent models, algorithms and applications*. Vol. 14. Springer Science & Business Media, 2012 (cit. on p. 5).
- [8] Hamilton Emmons and George Vairaktarakis. *Flow shop scheduling: theoretical results, algorithms, and applications*. Vol. 182. Springer Science & Business Media, 2012 (cit. on p. 6).

- [9] X. Li and M. Yin. «A discrete artificial bee colony algorithm with composite mutation strategies for permutation flow shop scheduling problem». In: *Scientia Iranica* 19.6 (2012), pp. 1921–1935. ISSN: 1026-3098. DOI: <https://doi.org/10.1016/j.scient.2012.10.034>. URL: <https://www.sciencedirect.com/science/article/pii/S1026309812002453> (cit. on p. 6).
- [10] Paz Perez-Gonzalez, Victor Fernandez-Viagas, and Jose M Framinan. «Permutation flowshop scheduling with periodic maintenance and makespan objective». In: *Computers & Industrial Engineering* 143 (2020), p. 106369 (cit. on p. 7).
- [11] Sigurdur Ólafsson. «Chapter 21 Metaheuristics». In: *Simulation*. Ed. by Shane G. Henderson and Barry L. Nelson. Vol. 13. Handbooks in Operations Research and Management Science. Elsevier, 2006, pp. 633–654. DOI: [https://doi.org/10.1016/S0927-0507\(06\)13021-2](https://doi.org/10.1016/S0927-0507(06)13021-2). URL: <https://www.sciencedirect.com/science/article/pii/S0927050706130212> (cit. on p. 10).
- [12] Dimitris Bertsimas and John Tsitsiklis. «Simulated annealing». In: *Statistical science* 8.1 (1993), pp. 10–15 (cit. on p. 10).
- [13] Peter JM Van Laarhoven and Emile HL Aarts. «Simulated annealing». In: *Simulated annealing: Theory and applications*. Springer, 1987, pp. 7–15 (cit. on p. 10).